# TÀI LIỆU ĐẶC TẢ KIẾN TRÚC PHẦN MỀM
# (Software Architecture Document – SAD)

## Hệ thống Secure Gateway mã hóa dữ liệu IP thời gian thực sử dụng STM32

## Phiên bản 1.0.0

**Được soạn bởi: Vũ Văn Tiến**

**Hà Nội, tháng 3, năm 2026**

# MỤC LỤC


1. GIỚI THIỆU ..... 1
1.1. Các thuật ngữ sử dụng trong tiêu chuẩn ..... 1
1.2. Phạm vi của tài liệu ..... 1
1.2.1. Mục đích và phạm vi của SAD ..... 1
1.2.2. Tổ chức tài liệu SAD ..... 1
1.2.3. Đại diện các bên liên quan ..... 2
1.2.4. Định nghĩa các Điểm nhìn ..... 2
2. MỤC TIÊU VÀ BỐI CẢNH ..... 3
2.1. Các yêu cầu dẫn dắt kiến trúc quan trọng ..... 3
2.2. Bối cảnh giải pháp ..... 4
2.2.1. Các cách tiếp cận kiến trúc ..... 4
2.2.2. Kết quả phân tích ..... 4
2.2.3. Mức độ đáp ứng yêu cầu từ SRS ..... 5
2.3. Quy ước đặt tên ..... 5
2.4. Tiêu chuẩn lập trình ..... 6
3. CÁC GÓC NHÌN KIẾN TRÚC ..... 7
3.1. Góc nhìn truyền thông ..... 7
3.1.1. Sơ đồ thực thể và luồng dữ liệu SPI-DMA ..... 7
3.1.2. Danh mục các phần tử kiến trúc ..... 9
3.1.3. Giao diện và hành vi ..... 9
3.2. Góc nhìn an ninh thông tin ..... 9
3.2.1. Luồng dữ liệu nhạy cảm và quy trình mã hóa AES ..... 9
3.2.2. Cơ chế bảo vệ khóa mã hóa ..... 10
3.3. Góc nhìn độ tin cậy ..... 10
3.3.1. Cơ chế giám sát Independent Watchdog (IWDG) ..... 10
3.3.2. Khả năng tự phục hồi lỗi kết nối vật lý ..... 11
3.3.3. Cơ chế xử lý lỗi ..... 11
3.4. Góc nhìn quy trình/đa nhiệm ..... 12
4. QUAN HỆ GIỮA CÁC GÓC NHÌN ..... 13
4.1. Mối quan hệ tổng quát giữa các góc nhìn ..... 13

**4.2. Ánh xạ giữa các thành phần phần mềm và phần cứng** ..... 13
**5. TÀI LIỆU THAM KHẢO** ..... 14
**6. TỪ ĐIỂN THUẬT NGỮ VÀ VIẾT TẮT** ..... 14
**7. PHỤC LỤC** ..... 15
**Phục lục A Sơ đồ kết phần cứng chi tiết** ..... 15
**A.1. Sơ đồ bố trí chân vi điều khiển** ..... 15
**A.2. Danh mục ánh xạ tín hiệu và chế độ cấu hình** ..... 16
**Phục lục B Phân bổ kênh DMA1 và mức độ ưu tiên** ..... 16
**Phục lục C Ước tính phân bổ bộ nhớ** ..... 17
**C.1. Phân bổ bộ nhớ RAM** ..... 17
**C.2. Phân bổ bộ nhớ chương trình** ..... 17

# MỤC LỤC HÌNH ẢNH

**Hình 3.1: Sơ đồ kiến trúc tổng thể** ..... 7
**Hình 3.1.1.1: Sơ đồ luồng dữ liệu** ..... 8
**Hình 3.2.1.1: Quy trình mã hóa AES** ..... 10
**Hình 7.A.1.1: Cấu hình các chân của vi điều khiển** ..... 16

# MỤC LỤC BẢNG

**Bảng 1.2.3.1: Bảng đại diện các bên liên quan** ..... 2
**Bảng 3.1.1.1: Bảng các kênh DMA1** ..... 9
**Bảng 3.3.1.1: Bảng cơ chế xử lý lỗi** ..... 12
**Bảng 3.4.1: Bảng phân phối mức độ ưu tiên của các tác vụ** ..... 13
**Bảng 4.2.1: Bảng ánh xạ các thành phần phần mềm với phần cứng** ..... 14
**Bảng 6.1: Bảng từ điển thuật ngữ và viết tắt** ..... 15
**Bảng 7.A.2.1: Chế độ cấu hình các chân của vi điều khiển** ..... 16
**Bảng 7.1.1: Bảng phân bổ kênh DMA1 và mức độ ưu tiên** ..... 17
**Bảng 7.C.1.1: Bảng ước tính bộ nhớ RAM** ..... 17
**Bảng 7.C.2.1: Bảng ước tính bộ nhớ Flash** ..... 18

# 1. GIỚI THIỆU

## 1.1. Các thuật ngữ sử dụng trong tiêu chuẩn

Tài liệu (Document): Tập hợp các dữ liệu mô tả kiến trúc phần mềm, không phân biệt phương tiên lưu trữ (ví dụ: file PDF, Word...). Giúp mọi người hiểu hệ thống phần mềm được thiết kế và tổ chức như thế nào.

Điểm nhìn (Viewpoint): Một khuôn mẫu hoặc bộ quy tắc để tạo ra một góc nhìn. Quy định các trình bày, ký hiệu và cách sử dụng để đảm bảo mọi góc nhìn đều nhất quán.

Góc nhìn (View): Là cách nhìn vào hệ thống từ một khía cạnh cụ thể, mỗi khía cạch này bao gồm một tập hợp các mối quan tâm (concerns).

Gói góc nhìn (View Packet): Đơn vị nhỏ nhất của tài liệu mô tả một phần cụ thể của hệ thống trong một góc nhìn.

## 1.2. Phạm vi của tài liệu

### 1.2.1. Mục đích và phạm vi của SAD

Tài liệu này đặc tả kiến trúc phần mềm cho dự án Secure Gateway AES-STM32, tập trung vào việc thực thi bảo mật tại tầng mạng (Network Layer) sử dụng mã hóa AES-128-CBC.

SAD đóng vai trò là cầu nối truyền đạt các ý tưởng, quyết định thiết kế cốt lõi đến lập trình viên, kỹ sư phần cứng và hội đồng nghiệm thu.

Các quyết định mang tính kiến trúc được trình bày bao gồm: Việc lựa chọn mô hình đa nhiệm FreeRTOS, cơ chế điều phối dữ liệu qua DMA và quản lý bộ nhớ tĩnh.

### 1.2.2. Tổ chức tài liệu SAD

Chương 1 – Giới thiệu: Trình bày mục đích, phạm vi, các đối tượng liên quan và định nghĩa cấu trúc của tài liệu.

Chương 2 – Mục tiêu và bối cảnh: Mô tả các mục tiêu và bối cảnh giúp định hình nên kiến trúc.

Chương 3 – Góc nhìn kiến trúc: Trình bày chi tiết các góc nhìn kiến trúc về truyền thông, an ninh thông tin, độ tin cậy và quy trình/đa nhiệm.

Chương 4 – Quan hệ giữa các góc nhìn: Phân tích sự nhất quán và mối quan hệ giữa các góc nhìn khác nhau.

Chương 5: Cung cấp các tài liệu tham khảo

Chương 6: Cung cấp từ điển thuật ngữ và viết tắt.

Chương 7: Cung cấp phục lục chi tiết.

### 1.2.3. Đại diện các bên liên quan

Kiến trúc này được thiết kế để giải quyết các nhu cầu và mối quan tâm cụ thể của các đối tượng sau:

| Vai trò | Mối quan tâm chính |
|---|---|
| Kỹ sư phần mềm | Cấu trúc Task trong FreeRTOS, giao diện hàng đợi (Queue) và cách tích hợp thư viện Tiny-AES. |
| Kỹ sư phần cứng | Tương tác SPI giữa STM32 và ENC28J60, cấu hình DMA và dòng tải vật lý. |
| Kỹ sư bảo mật | Tính bảo mật của khóa AES lưu trong Flash và luồng dữ liệu bản rõ (plaintext) trong RAM. |
| Kỹ sư bảo trì hệ thống | Sự rõ ràng trong cấu trúc mã nguồn, hệ thống quy ước đặt tên và các tiêu chuẩn lập trình được áp dụng. |
| Hội đồng nghiệm thu | Khả năng đáp ứng thời gian thực (real-time), thông lượng dữ liệu mạng và độ tin cậy của hệ thống. |

*Bảng 1.2.3.1: Bảng đại diện các bên liên quan*

### 1.2.4. Định nghĩa các Điểm nhìn

Hệ thống sử dụng các điểm nhìn sau để thực hiện phân tích và thiết kế kiến trúc:

Điểm nhìn truyền thông: Tập trung vào các con đường truyền dẫn dữ liệu và các thành phần phần mềm/phần cứng trực tiếp tham gia vào việc gửi và nhận thông tin.

Điểm nhìn an ninh thông tin: Tập trung vào luồng (flow) và vị trí lưu trữ của các thông tin nhạy cảm cần được bảo vệ. Xác định các phần tử thực hiện dịch vụ bảo mật và các biện pháp bảo vệ dữ liệu khỏi bị rò rỉ hoặc tấn công.

Điểm nhìn độ tin cậy: Mô tả các chức năng quan trọng, các cơ chế giúp hệ thống duy trì hoạt động, tự phục hồi khi xảy ra lỗi và cách xử lý khi gặp lỗi.

Điểm nhìn quy trình/đa nhiệm: Phân tích cách thức dữ liệu được cung cấp, sử dụng, lưu trữ và cách hệ thống xử lý khi khối lượng dữ liệu tăng cao.

# 2. MỤC TIÊU VÀ BỐI CẢNH

## 2.1. Các yêu cầu dẫn dắt kiến trúc quan trọng

Kiến trúc phầm mềm được định hình bởi các yêu cầu hành vi và chất lượng:

Yêu cầu hành vi:

- Thực hiện định tuyến gói tin UDP theo địa chỉ IPv4 tĩnh giữa hai phân vùng mạng. Chỉ tiếp nhận và xử lý các gói tin UDP không phân mảnh (Fragment Offset = 0) và loại bỏ các giao thức khác để tiết kiệm tối đa không gian lưu trữ của bộ đệm RAM.
- Mã hóa và giải mã Payload của gói tin IP bằng thuật toán AES-128-CBC với khóa riêng biệt cho từng địa chỉ IP (Key_1 cho PC, Key_2 cho Raspberry Pi). Quá trình này bắt buộc sử dụng IV (Initialization Vector) cố định được cấu hình sẵn trong firmware nhằm đơn giản hóa việc triển khai).

Yêu cầu chất lượng:

- Đảm bảo khả năng xử lý ổn định 50 gói tin/giây với độ trễ không vượt quá 10ms cho mỗi gói tin có kích thước tối đa 598 bytes.

- Vận hành ổn định ổn định trong giới hạn bộ nhớ hạn chế của vi điều khiển STM32F103C8T6 với 20KB RAM và 64KB Flash.
- Khả năng tự phục hồi thông qua Watchdog Timer (khi hệ thống bị treo trong >= 1000ms) và xử lý lỗi kết nối vật lý giữa vi điều khiển với module ENC28J60.

## 2.2. Bối cảnh giải pháp

### 2.2.1. Các cách tiếp cận kiến trúc

Mô hình đa nhiệm dựa trên FreeRTOS: Sử dụng RTOS để quản lý đồng thời việc thu nhận/truyền dữ liệu mạng, mã hóa và giám sát hệ thống, đảm bảo tính đáp ứng thời gian thực.

Kiến trúc song song hóa SPI: Sử dụng hai bộ ngoại vi SPI độc lập kết nối với hai module ENC28J60 để tối ưu hóa thông lượng và giảm nhiễu chéo giữa hai luồng dữ liệu.

Cơ chế đẩy dữ liệu trực tiếp: Áp dụng DMA để di chuyển dữ liệu giữa SPI và RAM mà không cần sự can thiệp trực tiếp của CPU, giúp giải phóng tài nguyên để thực hiện thuật toán AES.

Quản lý bộ đệm tĩnh: Sử dụng Static Buffer Pool và hàng đợi FreeRTOS để tránh phân mảnh RAM và đảm bảo hệ thống không vượt qua ngưỡng 75% bộ nhớ khả dụng.

### 2.2.2. Kết quả phân tích

Các phân tích định lượng cho thấy kiến trúc được thiết kế hoàn toàn phù hợp với mục đích sử dụng ứng dụng:

- Phân tích độ trễ: Tổng thời gian sử lý của một gói tin (Bao gồm truyền/nhận SPI, mã hóa AES và tính toán lại Checksum) ước tính chỉ mất khoảng ~5ms, đáp ứng tốt yêu cầu độ trễ < 10ms.
- Phân tích thông lượng: Với tốc độ xử lý 50 gói tin/gây, hệ thống đạt thông lượng thực tế ~239 Kbps (thấp hơn chuẩn 10BASE-T với thông lượng tối đa là 10 Mbps), đủ khả năng hoạt động với các tác vụ bảo mật tầng mạng cơ bản.

- Phân tích bộ nhớ: Tổng dung lượng RAM chiếm dụng dự kiến ở mức 15KB (~75%), cung cấp đầy đủ không gian an toàn cho các hoạt động ngắt và ngăn ngừa lỗi Stack Overflow.

### 2.2.3. Mức độ đáp ứng yêu cầu từ SRS

Kiến trúc này bao gồm đầy đủ các yêu cầu chức năng đã được đặc tả trong tài liệu SRS:

- Chức năng [FR-01] và [FR-03] được đảm bảo bởi thiết kế Driver cho module ENC28J60, kiến trúc DMA, RX_SPI1_Task và RX_SPI2_Task.
- Chức năng phân tích [FR-02], mã hóa [FR-04] và đóng gói [FR-05] được thực thi thông qua lớp xử lý gồm Packet_Processing_Task được tích hợp thư viện Tiny-AES.
- Cơ chế chuyển tiếp [FR-06] và quản lý hàng đợi [FR-07] được thực hiện qua hàng đợi FreeRTOS kết hợp Static Buffer Pool để chống phân mảnh, bao gồm cả cơ chế giải phóng bộ đệm khi phát hiện kích thước gói tin vượt qua giới hạn hoặc hàng đợi quá tải.
- Cơ chế giải mã [FR-08] được thực hiện trên thiết bị đầu cuối, hoàn toàn độc lập và nằm ngoài phạm vi đặc tả kiến trúc Firmware của tài liệu này.

## 2.3. Quy ước đặt tên

Nhằm đảm bảo tính nhất quán và giúp người bảo trì dễ dàng hiểu mã nguồn, hệ thống tuân thủ các quy tắc đặt tên sau:

Tên tệp tin: Sử dụng chữ cái in thường, phân tách bằng dấu gạch dưới (ví dụ: enc28j60_driver.c)

Tác vụ FreeRTOS: Bắt đầu bằng tiền tố v (đại diện cho kiểu dữ liệu void) và kết thúc bằng _Task (ví dụ: vRX_Task, vPacket_Processing_Task).

Hàng đợi và Semaphore: Sử dụng tiền tố x (đại diện cho các kiểu dữ liệu không phải kiểu cơ bản) (ví dụ: xRX_Queue).

Biến ngoại vi: Sử dụng tiền tố h (handle) theo chuẩn HAL (ví dụ: hspi1, hdma_spi1_rx).

Hằng số và Macro: Sử dụng chữ in hoa toàn bộ, phân tách bằng dấu gạch dưới (ví dụ: MAX_PAYLOAD_SIZE, AES_KEY_LENGTH).

Biến cục bộ: Tên biến được cấu tạo từ các từ tiếng anh viết thường, ngăn cách bởi dấu gạch dưới (ví dụ: buffer_index...).

## 2.4. Tiêu chuẩn lập trình

Các tiêu chuẩn sau sẽ được áp dụng để đảm bảo hiệu suất tối ưu trên vi điều khiển STM32 và sự an toàn của hệ thống thời gian thực:

Ngôn ngữ lập trình: Sử dụng ngôn ngữ C (chuẩn C99) để đảm bảo tính tương thích và hiệu năng.

Quản lý bộ nhớ: Không sử dụng cấp phát bộ nhớ động (malloc, free) nhằm tránh phân mảnh RAM và lỗi tràn bộ nhớ trong điều kiện tài nguyên hạn chế 20 KB. Toàn bộ bộ đệm gói tin phải được cấp phát tĩnh trong Static Bufer Pool.

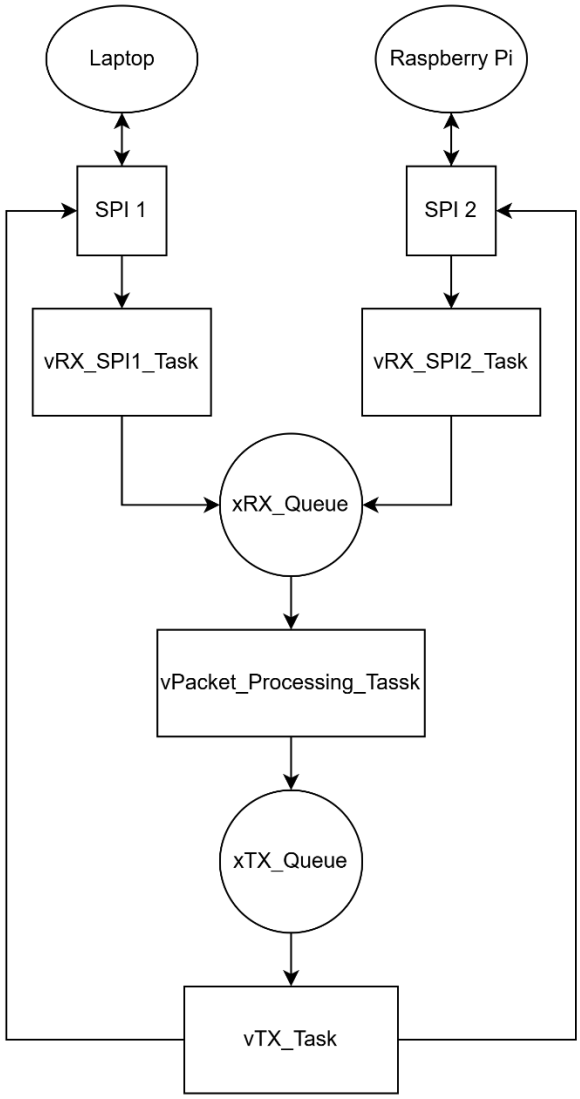
Xử lý ngắt: Các hàm ngắt phải được giữ ở mức ngắn gọn nhất có thể. Mọi xử lý logic nặng phải được đẩy sang các Task của FreeRTOS thông qua Semaphore hoặc Queue.

Kiểm tra lỗi: Mọi thao tác đọc/ghi vào ENC28J60 qua SPI phải được kiểm tra giá trị trả về hoặc trạng thái phản hồi để đảm bảo hệ thống không bị treo ngoại vi.

Chú thích mã nguồn: Mỗi hàm phải có tiêu đề mô tả mục đích, tham số đầu vào và giá trị trả về.

Kỹ thuật Zero-copy: Quá trình chuyển gói tin giữa các tác vụ (Tasks) thông qua Queue bắt buộc phải sử dụng phương pháp truyền con trỏ (tham chiếu) thay vì sao chép toàn bộ khối dữ liệu, nhằm tối ưu hóa tốc độ và bảo vệ không gian RAM.

# 3. CÁC GÓC NHÌN KIẾN TRÚC

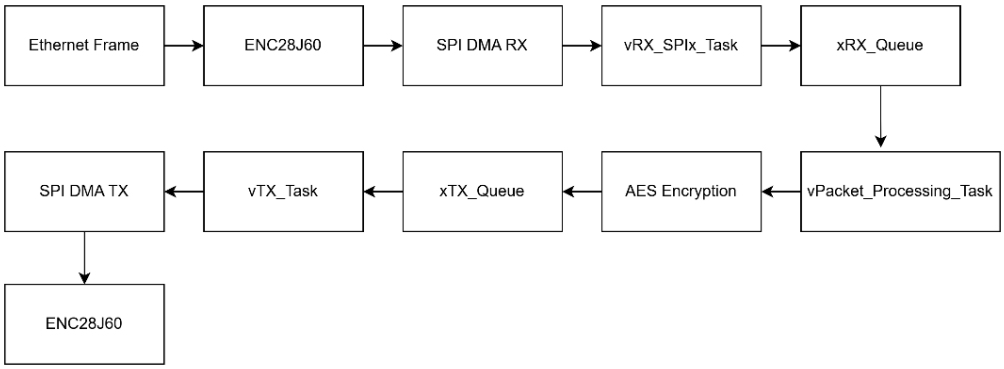



*Hình 3.1: Sơ đồ kiến trúc tổng thể*

## 3.1. Góc nhìn truyền thông

Góc nhìn này tập trung vào đường truyền vật lý và logic để đảm bảo thông lượng mạng đạt mức tối ưu.

### 3.1.1. Sơ đồ thực thể và luồng dữ liệu SPI-DMA




*Hình 3.1.1.1: Sơ đồ luồng dữ liệu*

Luồng nhận dữ liệu sẽ thực hiện việc xử lý gói tin theo các bước lần lượt dưới đây:

- Module ENC28J60 nhận gói tin Ethernet từ mạng.
- Khi gói tin được lưu trong bộ đệm nội bộ của ENC28J60, module sẽ kích hoạt tín hiệu ngắt ngoài (EXTI) tới STM32.
- STM32 sẽ khởi động DMA1 (Channel 2/4) để đọc dữ liệu gói tin từ ENC28J60 qua giao tiếp SPI vào RAM buffer của STM32.
- Sau khi DMA hoàn tất truyền dữ liệu, địa chỉ gói tin trong bộ đệm sẽ được đưa vào xRX_Queue.

Luồng gửi dữ liệu sẽ thực hiện việc xử lý gói tin theo các bước lần lượt dưới đây:

- Sau khi xử lý và mã hóa, địa chỉ của gói tin trong bộ đệm sẽ được đưa vào xTX_Queue.
- vTX_Task lấy một phần tử từ xTX_Queue và tiến hành xử lý.
- STM32 kích thoạt DMA1 (Channel 3/5) để truyền dữ liệu từ bộ đệm của STM32 tới ENC28J60 thông qua SPI.
- Sau khi DMA hoàn thất, ENC28J60 sẽ đưa gói tin ra Ethernet network.

| DMA1 | | Chức năng |
|---|---|---|
| SPI 1 | DMA1 Channel 2 | Luồng nhận (RX) |
| | DMA1 Channel 3 | Luồng gửi (TX) |
| SPI 2 | DMA1 Channel 4 | Luồng nhận (RX) |
| | DMA1 Channel 5 | Luồng gửi (TX) |

*Bảng 3.1.1.1: Bảng các kênh DMA1*

### 3.1.2. Danh mục các phần tử kiến trúc

Master Controller (STM32F103C8T6): Điều phối toàn bộ quá trình giao tiếp và xử lý dữ liệu.

Network Slaves (ENC28J60): Chịu trách nhiệm chuyển đổi tín hiệu Ethernet sang SPI và ngược lại.

DMA Controller: Đóng vai trò là cầu nối vận chuyển dữ liệu giữa ngoại vi (Module ENC28J60) và bộ nhớ.

### 3.1.3. Giao diện và hành vi

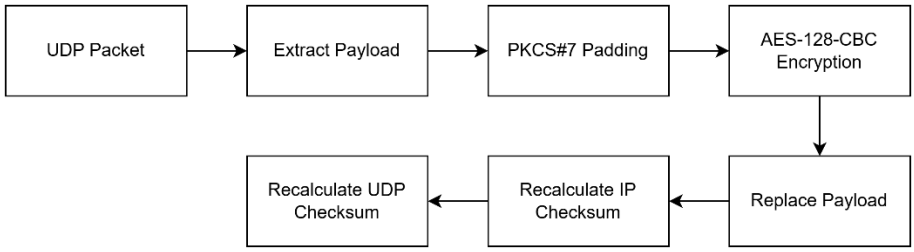
Giao diện SPI: Hoạt động ở chế độ Master (STM32F103C8T6) với tần số xung nhịp cho cả hai bộ SPI là 18MHz để đảm bảo thông lượng.

Hành vi: Khi có gói tin, CPU chỉ can thiệp ở bước cấu hình DMA, sau đó DMA tự thực hiện việc sao chép dữ liệu từ ENC28J60 sang bộ đệm, khi đó thì CPU sẽ đi làm các công việc khác (ví dụ: thực hiện việc mã hóa AES cho Payload).

## 3.2. Góc nhìn an ninh thông tin

Góc nhìn này đặc tả cách thức bảo mật dữ liệu Payload khi đi qua Gateway.

### 3.2.1. Luồng dữ liệu nhạy cảm và quy trình mã hóa AES




*Hình 3.2.1.1: Quy trình mã hóa AES*

Tiếp nhận: Gói tin bản rõ (Plaintext) được bóc tách tại tầng mạng (Network Layer).

Mã hóa: Bóc tách gói tin để lấy phần Payload, sử dụng cơ chế PKCS#7 Padding để thêm các bytes vào cuối của phần Payload sao cho tổng số byte của phần Payload là bội số của 16. Sử dụng thuật toán AES-128-CBC để mã hóa phần Payload.

Đầu ra: Gói tin sau khi mã hóa (Ciphertext) được tính toán lại Checksum và đóng gói lại IP/UDP header mới trước khi chuyển tiếp.

### 3.2.2. Cơ chế bảo vệ khóa mã hóa

Lưu trữ khóa: Khóa mã hóa (AES Keys) được nhúng vào vùng nhớ Flash nội bộ của vi điều khiển. Sử dụng 2 key khác nhau cho 2 địa chỉ IP và dùng IV (Initialization Vector) cố định, giúp việc thực hiện dễ dàng hơn.

Bảo mật: Khóa mã hóa này sẽ không bao giờ được gửi qua mạng sau khi biên dịch.

## 3.3. Góc nhìn độ tin cậy

Góc nhìn này sẽ mô tả các cơ chế đảm bảo hệ thống vận hành liên tục và ổn định.

### 3.3.1. Cơ chế giám sát Independent Watchdog (IWDG)

Hệ thống sử dụng cơ chế Check-in Bitmask phối hợp với nhân FreeRTOS. Mỗi tác vụ quan trọng bắt buộc phải báo cáo trạng thái định kỳ về tác vụ giám sát thông qua cơ chế cờ tập trung. Chỉ khi tất các các

tác vụ xác nhận hoạt động bình thường, lệnh làm mới (refresh) IWDG mới được thực thi. Cơ chế này cho phép phát hiện lỗi treo ở mức độ từng tác vụ đơn lẻ, đảm bảo độ tin cậy cao.

Mã giả:

```
Task1 → set bit0
Task2 → set bit1
Task3 → set bit2
if   bitmask == 0b111
        refresh watchdog
else
        reset system
```

### 3.3.2. Khả năng tự phục hồi lỗi kết nối vật lý

Hệ thống sẽ tiến hành đọc thanh ghi PHSTAT1 của module ENC28J60 (đây là thanh ghi trạng thái của lớp vật lý). Tiến hành kiểm tra bit LLSTAT (PHY Latching Link Status bit) – bit này tự động lên 1 nếu có tín hiệu vật lý từ dây cáp mạng và xuống 0 nếu bị rút cáp hoặc module bị treo phần cứng.

Cứ mỗi 500ms, vHearbeat_Task sẽ đọc bit LLSTAT. Nếu phát hiện LLSTAT = 0 liên tục trong 4 giây (đã tính đến trường hợp nhiễu khi dây cáp vừa mới cắm vào và chưa kịp ổn định), hệ thống sẽ xác định là mất kết nối. Sử dụng chân PA2/PA3 (RST) để reset cứng module ENC28J60. Tiến hành chạy hàm khởi tạo và kiểm tra lại xem module đã hoạt động lại hay chưa.

### 3.3.3. Cơ chế xử lý lỗi

| Lỗi | Nguyên nhân | Cách xử lý |
|---|---|---|
| Kích thước gói tin quá lớn | Kích thước gói tin > 598 bytes. | Xóa gói tin. |

| | | |
|---|---|---|
| Hàng đợi đã tối đa | Kích thước hàng đợi nhỏ, tốc độ nhận gói quá nhanh hoặc có quá nhiều gói tin. | Xóa gói tin đang chờ được đưa vào hàng đợi. |
| ENC28J60 bị treo | SPI clock không ổn định, bộ đệm bị tràn. | Khởi động lại. |
| FreeRTOS bị bế tắc | Trạng thái chờ vô hạn khi lấy tài nguyên hoặc chờ sự kiện ngắt. | Watchdog reset. |
| Địa chỉ IP không hợp lệ | Địa chỉ IP không có trong bảng định truyến | Xóa gói tin. |

*Bảng 3.3.1.1: Bảng cơ chế xử lý lỗi*

## 3.4. Góc nhìn quy trình/đa nhiệm

Góc nhìn này phân tích cách thức FreeRTOS điều phối các tác vụ xử lý trong môi trường 20KB RAM.

| Tên tác vụ | Mức ưu tiên | Lý do kỹ thuật |
|---|---|---|
| vRX_SPIx_Task | 4 (Rất cao) | Đảm bảo phản ứng tức thời với ngắt phần cứng, ngăn chặn việc tràn bộ đệm tại module ENC28J60 và tối ưu hóa thông lượng mạng. |
| vTX_Task | 3 (Cao) | Đẩy dữ liệu ra mạng để giải phóng hàng đợi xTX_Queue, lấy chỗ cho gói tin mới. |
| vPacket_Processing_Task | 2 (Trung bình) | Thực hiện mã hóa AES-128-CBC tốn nhiều CPU (3.2 – 4.2ms). |
| vHeartbeat_Task | 1 (Thấp) | Giám sát IWDG và trạng thái kết nối, không đòi hỏi phản ứng tức thời. |

*Bảng 3.4.1: Bảng phân phối mức độ ưu tiên của các tác vụ*

# 4. QUAN HỆ GIỮA CÁC GÓC NHÌN

Chương này mô tả sự tương tác và nhất quán giữa các góc nhìn kiến trúc đã trình bày bên trên, đồng thời xác định cách thức các thành phần phần mềm được ánh xạ trực tiếp lên tài nguyên phần cứng của STM32F103C8T6.

## 4.1. Mối quan hệ tổng quát giữa các góc nhìn

Kiến trúc của hệ thống Secure Gateway là một khối thống nhất, trong đó các góc nhìn có mối quan hệ chặt chẽ với nhau.

Kiến trúc hệ thống lấy Góc nhìn quy trình/đa nhiệm (đặc tả tại mục 3.4) làm trọng tâm điều phối. Các tác vụ (Tasks) đóng vai trò thực thi logic, kết nối các ngoại vi phần cứng từ Góc nhìn Truyền thông (3.1) với các yêu cầu bảo mật từ Góc nhìn an ninh (3.2). Mọi hoạt động của các quy trình này đều được giám sát bởi cơ chế tại Góc nhìn độ tin cậy (3.3). Mọi góc nhìn đều phải tuân thủ các giới hạn về bộ dệm và hàng đợi đểm đảm bảo dữ liệu không vượt quá 15KB RAM.

## 4.2. Ánh xạ giữa các thành phần phần mềm và phần cứng

Để hệ thống vận hành thời gian thực, các thực thể phần mềm (Tasks) được ánh xạ cố định vào các tài nguyên phần cứng cụ thể dưới đây:

| Thành phần phần mềm | Tài nguyên phần cứng | Vai trò kiến trúc |
|---|---|---|
| vRX_SPI1_Task | SPI1, DMA1_Channel 2, EXTI0 (PA0). | Nhận gói tin từ Module ENC28J60 1. |
| vRX_SPI2_Task | SPI2, DMA1_Channel 4, EXTI1 (PA1) | Nhận gói tin từ Module ENC28J60 2. |
| vTX_SPI2_Task | SPI1, SPI2, DMA1_Channel 3, DMA1_Channel 5 | Gửi gói tin ra Module ENC28J60 1 và 2. |
| vPacket_Processing_Task | CPU Cortex-M3 (72MHz) | Bóc tách gói tin, mã hóa AES-128-CBC, |

| | | |
|---|---|---|
| | | tính toán lại Checksum, đóng gói lại gói tin. |
| vHeartbeat_Task | IWDG, GPIO (LED) | Giám sát trạng thái và báo hiệu. |
| Static Buffer Pool | SRAM (20KB) | Lưu trữ 10 bộ đệm gói tin cố định. |

*Bảng 4.2.1: Bảng ánh xạ các thành phần phần mềm với phần cứng*

# 5. TÀI LIỆU THAM KHẢO


[1] NoComplexity, “SAD Template – Architecture Playbook,” [Online]. Available: SAD Template — Architecture Playbook Jan 1 2026.

[2] STMicroelectronics, "STM32F103x8 STM32F103xB Datasheet,", Rev. 20, 2025.

[3] STMicroelectronics, "RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM®-based 32-bit MCUs," Rev. 21, 2021.

[4] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," FIPS PUB 197-upd1, May 9 2023.

[5] Microchip Technology Inc., "ENC28J60 Data Sheet: Stand-Alone Ethernet Controller with SPI Interface," 2012.

[6] Richard Barry and The FreeRTOS Team, "Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide," Release Version 1.1.0.

[7] Information Sciences Institute, "Internet Protocol (RFC 791)," University of Southern California, September 1981.


# 6. TỪ ĐIỂN THUẬT NGỮ VÀ VIẾT TẮT

Phần này cung cấp định nghĩa cho các thuật ngữ và từ viết tắt chuyên môn được sử dụng trong suốt tài liệu SAD.

| Thuật ngữ/ Viết tắt | Định nghĩa/ Ý nghĩa |
|---|---|

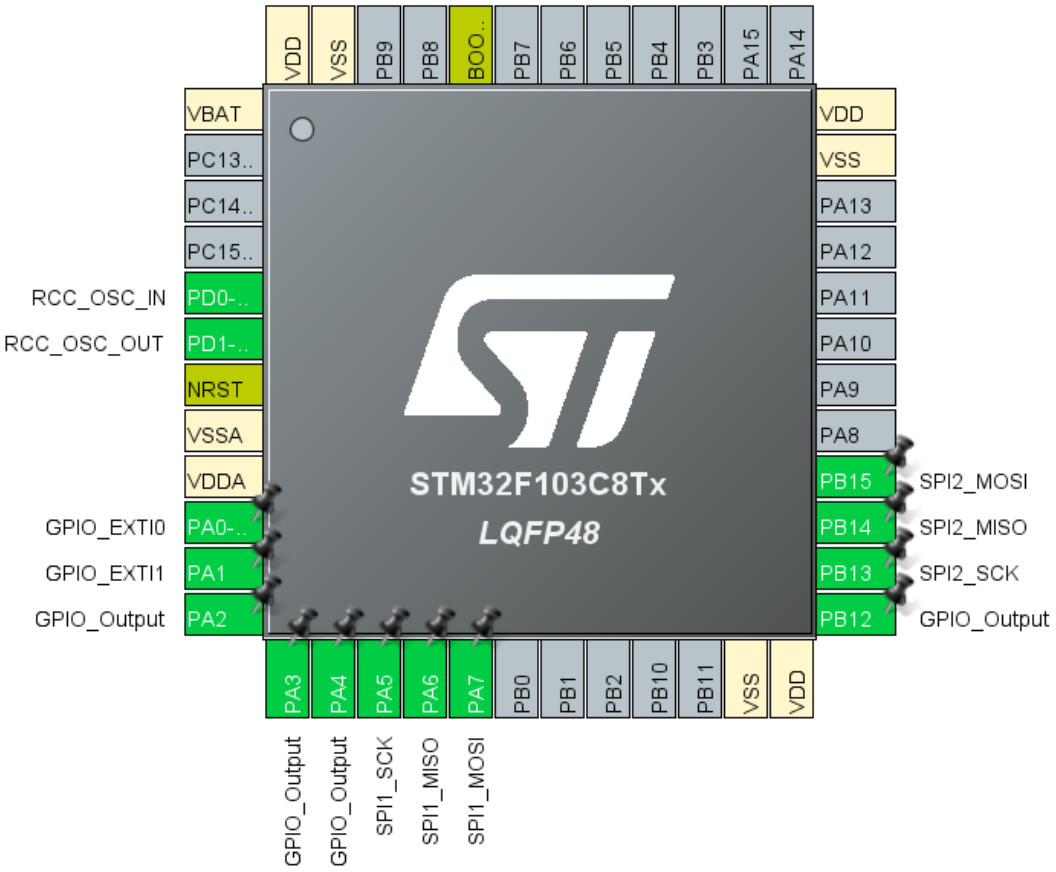
| | |
|---|---|
| SAD (Software Architecture Document) | Tài liệu mô tả kiến trúc phần mềm, trình bày các quyết định thiết kế quan trọng và các góc nhìn của hệ thống. |
| FreeRTOS | Hệ điều hành thời gian thực sử dụng để quản lý đa nhiệm và điều phối các tác vụ trên STM32F103C8T6. |
| DMA (Direct Memory Access) | Cơ chế cho phép ngoại vi truyền dữ liệu trực tiếp vào RAM mà không cần sự can thiệp liên tục của CPU. |
| ENC28J60 | Chip điều khiển Ethernet chuẩn 10BASE-T giao tiếp qua SPI. |
| IWDG (Independent Watchdog) | Bộ giám sát độc lập dùng để khởi động lại hệ thống nếu phần mềm bị treo hoặc lỗi. |
| Static Buffer Pool | Vùng bộ nhớ RAM được cấp phát tĩnh để cấu hình lưu trữ dữ liệu gói tin, giúp tránh phân mảnh bộ nhớ |
| Zero-copy | Kỹ thuật lập trình giúp xử lý dữ liệu thông qua con trỏ thay vì sao chép dữ liệu giữa các vùng nhớ để tối ưu tốc độ. |
| AES-128 | Chế độ mã hóa khối dùng thuật toán AES với khóa dài 128 bit. |
| CBC mode | Block sau phụ thuộc vào block trước đó, mỗi block có kích thước 16 bytes. Cần IV (Initialization Vector) |
| PKCS#7 Padding | Cơ chế bổ sung dữ liệu giả ở cuối đểm đảm bảo độ dài bản rõ (plaintext) luôn là bội số của 16 byte. |

*Bảng 6.1: Bảng từ điển thuật ngữ và viết tắt*

# 7. PHỤC LỤC

## Phục lục A Sơ đồ kết phần cứng chi tiết

### A.1. Sơ đồ bố trí chân vi điều khiển




*Hình 7.A.1.1: Cấu hình các chân của vi điều khiển*

## A.2. Danh mục ánh xạ tín hiệu và chế độ cấu hình

| Tín hiệu | Chân SPI1 | Chân SPI2 | Chế độ câu hình |
|---|---|---|---|
| SCK | PA5 | PB13 | Alternate Function Output Push-pull |
| MISO | PA6 | PB14 | Input with pull-up / pull-down |
| MOSI | PA7 | PB15 | Alternate function output Push-pull |
| NSS | PA4 | PB12 | General purpose output push-pull |
| INT | PA0 (EXTI0) | PA1 (EXTI1) | Input with pull-up / pull-down |
| RST | PA2 | PA3 | General purpose output push-pull |

*Bảng 7.A.2.1: Chế độ cấu hình các chân của vi điều khiển*

## Phục lục B Phân bổ kênh DMA1 và mức độ ưu tiên

| Yêu cầu DMA | Kênh | Hướng | Mức ưu tiên |
|---|---|---|---|
| SPI1_RX | DMA1 Channel 2 | Ngoại vi đến bộ nhớ | Rất cao |
| SPI1_TX | DMA1 Channel 3 | Bộ nhớ đến ngoại vi | Cao |
| SPI2_RX | DMA1 Channel 4 | Ngoại vi đến bộ nhớ | Rất cao |
| SPI2_TX | DMA1 Channel 5 | Bộ nhớ đến ngoại vi | Cao |

*Bảng 7.1.1: Bảng phân bổ kênh DMA1 và mức độ ưu tiên*

## Phục lục C Ước tính phân bổ bộ nhớ

### C.1. Phân bổ bộ nhớ RAM

Hệ thống được thiết kế để vận hành ổn định ở mức sử dụng 15KB RAM, dư 5KB phòng cho các hoạt động ngắt, bổ sung khi thiếu và tránh lỗi tràn ngăn xếp (Stack Overflow).

| Thành phần | Chi tiết | Dung lượng ước tính |
|---|---|---|
| Static Buffer Pool | 10 bộ đệm x 600 bytes (cho gói tin UDP/IP) | 6.0KB |
| Task Stacks (5 Task người dùng, 2 Task hệ thống (Idle và Timer Task) | 7 Tasks x 1024 bytes/task (mỗi Task sẽ dử dụng khoảng 1KB) | 7.0KB |
| FreeRTOS Heap & Queue | Quản lý 2 hàng đợi (xRX_Queue, xTX_Queue) và Semaphores | 0.5KB |
| HAL & Global Variables | Các biến handle, biến trạng thái hệ thống | 1.5KB |
| Tổng cộng chiếm dụng | | 15.0KB |

*Bảng 7.C.1.1: Bảng ước tính bộ nhớ RAM*

### C.2. Phân bổ bộ nhớ chương trình

| Thành phần | Chi tiết | Dung lượng ước tính |
|---|---|---|
| FreeRTOS Kernel | Nhân hệ điều hành và các hàm điều phối | 12.0KB |
| STM32 HAL Drivers | Driver cho SPI, DMA, GPIO | 18.0KB |

| | | |
|---|---|---|
| Tiny-AES & Logic | Thuật toán AES-128-CBC và logic xử lý gói tin | 8.0KB |
| ENC28J60 Driver | Trình điều khiển module mạng | 6.0KB |
| Tổng cộng chiếm dụng | | 44.0KB |

*Bảng 7.C.2.1: Bảng ước tính bộ nhớ Flash*